

Guía de Implementación

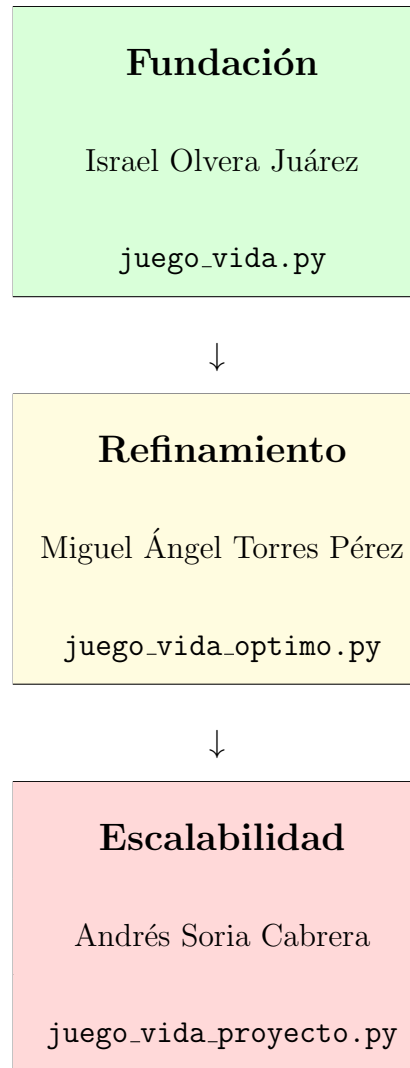
Juego de la Vida de Conway

Arquitectura en 3 capas

Capa Fundación	Israel Olvera Juárez
Capa Refinamiento	Miguel Ángel Torres Pérez
Capa Escalabilidad	Andrés Soria Cabrera

Programación Paralela y Concurrente
Universidad Nacional Autónoma de México

Mayo 2026



1. Visión general de la arquitectura

1.1. Las 3 capas

El proyecto está organizado en tres capas que se construyen una sobre otra:

Capa 1: Fundación juego_vida.py	→	GUI Pygame, Set[Cell], vecindario de Moore, algoritmo de 2 pasadas
Capa 2: Refinamiento juego_vida_optimo.py	→	Métodos puros, API programable, catálogo de 21 patrones, serialización YAML
Capa 3: Escalabilidad juego_vida_proyecto.py	→	Multiprocessing, Pool.map, 11 reglas B/S configurables, cámara y renderizado

1.2. Archivos del proyecto

Archivo	Tipo	Propósito
config.yml	Compartido	Dimensiones de grilla, colores, regla por defecto
requirements.txt	Compartido	Dependencias (pygame, numpy, PyYAML, scipy, etc.)
seed_patterns.yml	Fundación	9 patrones en matriz binaria (teclas 1–9)
juego_vida.py	Fundación	Implementación base con GUI Pygame
patterns.py	Refinamiento	Catálogo de 21 patrones + funciones de serialización
juego_vida_optimo.py	Refinamiento	Núcleo algorítmico puro, sin GUI
automata.py	Escalabilidad	Motor de reglas B/S con 11 presets
juego_vida_proyecto.py	Escalabilidad	Versión paralela con multiprocessing

1.3. Configuración central: config.yml

Todas las implementaciones leen este archivo. Los campos principales son:

1.4. Dependencias entre módulos

config.yml < leído por los 3 .py

Campo	Valor	Descripción
width / height	800	Tamaño de ventana en píxeles
cell_size	16	Píxeles por célula (50×50 celdas)
grid_is_infinite	True	Wrapping toroidal con operador %
rule_string	B3/S23	Regla de autómatas por defecto
sleep	0.15	Segundos entre generaciones (solo v1)
fps	40	Fotogramas por segundo
reproduction		
underpopulation	3/2/3	B3/S23 clásico (legado)
overpopulation		

`automata.py` (sin dependencias del proyecto)

`patterns.py` (sin dependencias del proyecto)

`juego_vida.py`
(GUI Pygame)

`juego_vida_optimo.py`
(headless)

`juego_vida_proyecto.py`
(multiprocessing)

1.5. Tipo común

El tipo `Cell = Tuple[int, int]` representa una coordenada (`columna`, `fila`) en la grilla. Aparece en `patterns.py` y es usado por todas las implementaciones.

2. Capa Fundación – Israel Olvera Juárez

2.1. Archivos asignados

- `juego_vida.py` – implementación base con GUI Pygame (259 líneas)
- `seed_patterns.yml` – 9 patrones en formato de matriz binaria

2.2. Clase GameOfLife

La clase `GameOfLife` contiene toda la lógica del juego y el renderizado.

2.2.1. Constructor

```
def __init__(self) -> None:
    self._config = self._get_config()           # lee config.yml
    self._n_cols = 800 // 16                    # = 50 columnas
    self._n_rows = 800 // 16                   # = 50 filas
    self._living_cells: Set[Cell] = set()       # conjunto disperso
    self._is_paused = True                     # empieza en pausa
    self._rule = get_rule("B3/S23")            # regla de Conway
```

2.2.2. La decisión de diseño más importante: Set[Cell]

En vez de una matriz 2D de $50 \times 50 = 2,500$ celdas, solo se almacenan las coordenadas de las células **vivas**. Esto hace que:

- El paso de simulación sea $O(vivas + vecinas)$ en vez de $O(50 \times 50)$
- En grillas dispersas (pocas células) sea extremadamente rápido
- La memoria usada sea mínima

Una célula viva en la columna 5, fila 10 es simplemente la tupla (5, 10). Una célula muerta **no se almacena**; se asume muerta si no está en el conjunto.

2.2.3. Algoritmo de 2 pasadas – `run_logic()`

Este es el corazón del juego. Cada generación se calcula en dos pasos:

1. **Paso 1 – Supervivencia:** para cada célula viva, contar sus 8 vecinos. Si `should_survive(vecinos)` es True, se mantiene viva. Además, se recolectan todos los vecinos de células vivas en `all_neighbors`.

2. **Paso 2 – Reproducción:** para cada vecino de una célula viva que esté **muerto**, contar sus vecinos. Si `should_reproduce(vecinos)` es `True`, nace.

```
def run_logic(self) -> None:
    new_living_cells = set()
    all_neighbors = set()

    # Paso 1: supervivencia
    for cell in self._living_cells:
        n = self._count_living_neighbors(cell)
        if self._should_cell_survive(n):
            new_living_cells.add(cell)
        all_neighbors.update(self._get_neighbors(cell))

    # Paso 2: reproducción
    for cell in all_neighbors:
        if cell not in self._living_cells:
            n = self._count_living_neighbors(cell)
            if self._should_cell_reproduce(n):
                new_living_cells.add(cell)

    self._living_cells = new_living_cells
```

2.2.4. Vecindario de Moore con wrapping

Cada célula tiene 8 vecinos (horizontal, vertical y diagonal). Con `grid_is_infinite = True`, los bordes se envuelven usando el operador módulo (%), creando una grilla toroidal.

```
def _get_neighbors(self, cell: Cell) -> List[Cell]:
    col, row = cell
    neighbors = []
    for dc in [-1, 0, 1]:
        for dr in [-1, 0, 1]:
            if dc == 0 and dr == 0:
                continue
            nc = (col + dc) % self._n_cols      # wrapping
            nr = (row + dr) % self._n_rows
            neighbors.append((nc, nr))
    return neighbors
```

2.2.5. Controles de usuario

2.3. seed_patterns.yml

Contiene 9 patrones (blinker, glider, lwss, pulsar, gosper_glider_gun, toad, beacon, r_pentomino, diehard) en formato de matriz binaria. Cada patrón tiene un ID numérico que corresponde a la tecla que lo carga.

Ejemplo del formato:

```
- patterns:
  1:
```

Tecla / Acción	Método	Efecto
Espacio	–	Pausar / reanudar simulación
Flecha derecha	–	Avanzar 1 solo paso (en pausa)
C	<code>clear()</code>	Limpiar todas las células
G / 0	<code>_generate_random_init_grid()</code>	Grilla aleatoria (5 %–20 % densidad)
1–9	<code>_generate_pattern_from_catalog()</code>	Cargar patrón del catálogo
P	<code>_print_catalog()</code>	Imprimir catálogo en consola
R	<code>_cycle_rule()</code>	Cambiar regla de autómata
S	<code>_save_current_pattern()</code>	Guardar patrón a YAML
Click mouse	–	Alternar célula viva/muerta

```
name: "blinker"
matrix: [[0,1,0], [0,1,0], [0,1,0]]
```

La función `_generate_pattern_from_catalog(id_)` busca el nombre del patrón en un diccionario (`_PATTERN_KEY_MAP`) y lo carga del catálogo de `patterns.py`.

2.4. Puntos para exponer

1. **La representación Set[Cell]:** explicar por qué es más eficiente que una matriz. Solo guardamos lo que está vivo. Una grilla de 50×50 con 10 % de densidad almacena solo 250 tuplas en vez de 2,500 enteros.
2. **El algoritmo de 2 pasadas:** primero decidimos qué células vivas sobreviven, luego qué células muertas nacen. Es simple pero poderoso.
3. **El vecindario de Moore:** 8 vecinos, y con wrapping los bordes no existen. La grilla es un toroide.
4. **Demostrar en vivo:** ejecutar `python juego_vida.py`, cargar el blinker con la tecla 1, ver la oscilación.
5. **El seed_patterns.yml:** cómo las matrices binarias se convierten en conjuntos de coordenadas y se centran en la grilla.

3. Capa Refinamiento – Miguel Ángel Torres Pérez

3.1. Archivos asignados

- `juego_vida_optimo.py` – núcleo algorítmico puro, headless (203 líneas)
- `patterns.py` – catálogo de 21 patrones + serialización YAML (259 líneas)

3.2. Clase `GameOfLifeOptimo`

Toma la lógica de `GameOfLife` y la refactoriza para que sea más limpia, testeable y extensible. Las diferencias clave:

Aspecto	v1 (<code>juego_vida.py</code>)	v2 (<code>optimo.py</code>)
Líneas de código	259	203
Dependencias	pygame, numpy, yaml	Solo yaml
GUI	Sí (800×800)	No (headless)
Métodos puros	2 (parcial)	4 (completos)
Manejo de patrones	9 patrones, código repetido	21 patrones, API unificada
Reglas de autómata	1 (Conway)	11 (todas)
API pública	Solo <code>process_events/run_logic/draw</code>	10 métodos programables

3.2.1. Métodos puros

Un método puro es aquel que:

- No modifica el estado del objeto (sin efectos secundarios)
- Solo depende de sus parámetros de entrada
- Es **thread-safe**: puede ejecutarse en paralelo sin locks

```
def _count_living_neighbors(self, cell: Cell) -> int:
    """M TODO PURO: Sin efectos secundarios, thread-safe."""
    return sum(1 for n in self._get_neighbors(cell)
               if n in self._living_cells)

def _should_cell_survive(self, n: int) -> bool:
    """M TODO PURO: Encapsula reglas de supervivencia."""
    return self._rule.should_survive(n)

def _should_cell_reproduce(self, n: int) -> bool:
    """M TODO PURO: Encapsula reglas de reproducci n."""
    return self._rule.should_reproduce(n)
```


3.2.2. API programable

A diferencia de la versión base (que solo se controla con teclas y mouse), `GameOfLifeOptimo` expone una API que permite controlar todo desde código:

Método	Uso
<code>load_pattern("glider")</code>	Carga y centra un patrón del catálogo
<code>run_logic_step()</code>	Ejecuta 1 generación
<code>run_n_steps(500)</code>	Ejecuta N generaciones
<code>set_cells(cells)</code>	Establece directamente las células vivas
<code>get_cells()</code>	Retorna el estado actual
<code>clear()</code>	Vacía la grilla
<code>set_rule("highlife")</code>	Cambia la regla de autómeta
<code>list_pattern_names()</code>	Lista los 21 patrones disponibles
<code>patterns_by_type("spaceship")</code>	Filtra patrones por categoría

Ejemplo de uso:

```
g = GameOfLifeOptimo()
g.load_pattern("r_pentomino")
g.run_n_steps(100)
print(f"C lulas vivas: {len(g.get_cells())}")
g.set_rule("highlife")           # cambiar regla en caliente
```

3.3. patterns.py – Catálogo de patrones

3.3.1. Clase Pattern

```
@dataclass
class Pattern:
    name: str           # "glider"
    type: str           # "spaceship"
    cells: Set[Cell]    # {(1,0), (2,1), (0,2), (1,2), (2,2)}
    period: int | None  # 4
    description: str    # "La nave m s peque a..."
    author: str         # "John Conway"
```

Propiedades y métodos clave:

Método / Propiedad	Qué hace
<code>width, height</code>	Tamaño del bounding box
<code>bounds</code>	(min_col, min_row, max_col, max_row)
<code>to_matrix()</code>	Convierte Set[Cell] → matriz binaria normalizada
<code>centered(n_cols, n_rows)</code>	Centra el patrón en una grilla

Categoría	Cant.	Patrones
Still life	7	block, beehive, loaf, boat, tub, ship, pond
Oscilador	6	blinker, toad, beacon, pulsar, pentadecathlon, queen_bee
Nave espacial	4	glider, lwss, mwss, hwss
Cañón	1	gosper_glider_gun (período 30, 36 células)
Matusalén	4	r_pentomino, diehard, acorn, b_heptomino

3.3.2. Catálogo completo (21 patrones)

3.3.3. Funciones de serialización YAML

Convierten entre las tres representaciones del proyecto:

`Set[Cell] ← cells_to_matrix() / matrix_to_cells() → List[List[int]]`

`save_pattern_to_yaml(cells, name, path) → saved_patterns.yaml`

`load_patterns_from_yaml(path) ← saved_patterns.yaml`

```
def cells_to_matrix(cells: Set[Cell]) -> List[List[int]]:
    """Convierte conjunto disperso a matriz binaria normalizada."""
    min_c = min(c for c, _ in cells)
    min_r = min(r for _, r in cells)
    w = max(c for c, _ in cells) - min_c + 1
    h = max(r for _, r in cells) - min_r + 1
    matrix = [[0] * w for _ in range(h)]
    for c, r in cells:
        matrix[r - min_r][c - min_c] = 1
    return matrix

def save_pattern_to_yaml(cells, name, filepath):
    """Guarda un patrón a YAML con ID autoincremental."""
    matrix = cells_to_matrix(cells)
    data = load_existing_yaml(filepath)
    next_id = max(data['patterns'].keys()) + 1
    data['patterns'][next_id] = {'name': name, 'matrix': matrix}
    yaml.dump(data, open(filepath, 'w'))
```

3.4. Métricas de mejora

- -31 % líneas de código (259 → 203)
- -75 % dependencias externas (3 → 1)
- +100 % métodos puros (2 → 4)

- **+144 % patrones** soportados (9 → 21)
- **+1000 % reglas** de autómata (1 → 11)

3.5. Puntos para exponer

1. **Métodos puros:** qué son y por qué son el primer paso hacia el paralelismo. Un método sin efectos secundarios puede ejecutarse en cualquier orden, en cualquier hilo.
2. **API programable:** la versión base solo se usa con teclas. La versión refinada se puede controlar desde scripts, tests y otras aplicaciones.
3. **El catálogo de patrones:** 21 patrones con metadatos. Cada patrón sabe su tamaño, su período, su autor. Demo: cargar un glider, un pulsar, un gopher_glider_gun.
4. **Serialización YAML:** cómo un patrón dibujado con el mouse puede guardarse a disco y cargarse después. El flujo completo: Set[Cell] → matriz → YAML → disco.
5. **Métricas:** mostrar la tabla de mejora. Menos código, más funcionalidad.

4. Capa Escalabilidad – Andrés Soria Cabrera

4.1. Archivos asignados

- `juego_vida_proyecto.py` – versión paralela con `multiprocessing` (210 líneas)
- `automata.py` – motor de 11 reglas de autómatas configurables (65 líneas)

4.2. `juego_vida_proyecto.py` – Motor paralelo

4.2.1. ¿Por qué funciones sueltas y no una clase?

Python requiere que las funciones enviadas a otros procesos vía `multiprocessing` sean **serializables con pickle**. Las funciones definidas a nivel de módulo (no como métodos de clase) cumplen este requisito. Por eso `calcular_segmento` y `contar_vecinos` son funciones globales.

4.2.2. Conversión Set ↔ Grid

La versión paralela usa una matriz densa `List[List[int]]` de 200×200 en vez de `Set[Cell]`. Las funciones de conversión son:

```
def cells_set_to_grid(cells, filas, cols, offset_col=0, offset_row=0):
    """Set[Cell] -> matriz 2D con offset opcional."""
    tablero = [[0]*cols for _ in range(filas)]
    for c, r in cells:
        if 0 <= r+offset_row < filas and 0 <= c+offset_col < cols:
            tablero[r+offset_row][c+offset_col] = 1
    return tablero

def grid_to_cells_set(tablero):
    """Matriz 2D -> Set[Cell] (para guardar a YAML)."""
    cells = set()
    for r, row in enumerate(tablero):
        for c, val in enumerate(row):
            if val: cells.add((c, r))
    return cells
```

4.2.3. El worker: `calcular_segmento()`

Esta función es el núcleo del paralelismo. Cada proceso trabajador recibe una **franja de filas** y calcula la siguiente generación para esa franja.

```
def calcular_segmento(args):
    """Worker de multiprocessing. Calcula una franja de filas."""
    tablero, fila_inicio, fila_fin, filas_tot, cols_tot, born, survive = args
    segmento_nuevo = []
```

```

for fila in range(fila_inicio, fila_fin):
    nueva_fila = []
    for col in range(cols_tot):
        vecinos = contar_vecinos(tablero, col, fila, filas_tot,
                                cols_tot)
        estado_actual = tablero[fila][col]

        # Reglas de Conway con born/survive dinamicos
        if estado_actual == 1 and vecinos not in survive:
            nueva_fila.append(0)          # muere
        elif estado_actual == 0 and vecinos in born:
            nueva_fila.append(1)          # nace
        else:
            nueva_fila.append(estado_actual) # no cambia

    segmento_nuevo.append(nueva_fila)

return fila_inicio, fila_fin, segmento_nuevo

```

Datos clave del worker:

- Recibe born y survive como sets de enteros → reglas dinámicas
- Solo lee el tablero actual (nunca lo modifica) → sin condiciones de carrera
- Escribe exclusivamente en su propio segmento_nuevo → sin conflicto
- Usa contar_vecinos() con wrapping toroidal siempre activo

4.2.4. Pool.map() y descomposición por filas

El trabajo se divide en franjas horizontales, una por núcleo de CPU:

```

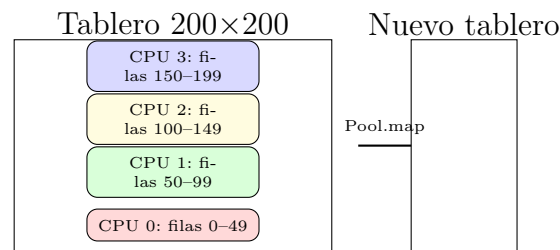
num_procesos = os.cpu_count() - 1      # Ej: 7 procesos en CPU de 8
nucleos
pool = mp.Pool(processes=num_procesos)

filas_por_proc = 200 // num_procesos  # Ej: 200/7 = 28 filas por proceso
tareas = []
for i in range(num_procesos):
    inicio = i * filas_por_proc
    fin = 200 if i == num_procesos-1 else (i+1) * filas_por_proc
    tareas.append((tablero, inicio, fin, 200, 200, rule.born, rule.
                    survive))

# EJECUCI N PARALELA
resultados = pool.map(calcular_segmento, tareas)

# REENSAMBLADO
nuevo_tablero = crear_tablero_vacio()
for inicio, fin, segmento in resultados:
    nuevo_tablero[inicio:fin] = segmento
tablero = nuevo_tablero

```



4.2.5. Cámara y renderizado

- **Zoom:** rueda del mouse, 4px – 60px por célula
- **Paneo:** teclas WASD o flechas
- **Renderizado selectivo:** solo se dibujan las celdas visibles
- **Grilla:** se oculta con $\text{zoom} \leq 5\text{px}$ por rendimiento
- **Pintado:** click izq = vivir, click der = matar (solo en pausa)
- Las coordenadas del mouse se ajustan por cámara y zoom

4.3. automata.py – Motor de reglas

4.3.1. Clase AutomataRule

```
@dataclass
class AutomataRule:
    born: Set[int]          # Ej: {3} para Conway
    survive: Set[int]       # Ej: {2, 3} para Conway
    name: str               # "Conway's Life"

    def should_survive(self, n: int) -> bool:
        return n in self.survive

    def should_reproduce(self, n: int) -> bool:
        return n in self.born

    @property
    def rule_string(self) -> str:
        return f"B{born}/S{survive}" # "B3/S23"
```

4.3.2. 11 presets

4.3.3. Parser de notación B/S

```
def parse_rule(rule_string: str) -> AutomataRule:
    """'B3/S23' -> AutomataRule(born={3}, survive={2,3})"""
    parts = rule_string.upper().split("/")
    born = set()
```

Preset	Regla	Comportamiento
conway	B3/S23	Clásico de Conway: gliders, osciladores, cañones
highlife	B36/S23	Parecido a Conway pero con replicador natural
seeds	B2/S	Todo es caótico, los patrones mueren rápido
maze	B3/S12345	Genera estructuras tipo laberinto
daynight	B3678/S34678	Simétrico: invertir vivo/muerto da igual
replicator	B1357/S1357	Patrones que se copian a sí mismos
morley	B368/S245	Naves de alta velocidad y explosiones
diamoeba	B35678/S5678	Formas ameboides que se expanden
2x2	B36/S125	Dominado por bloques 2×2
34life	B34/S34	Explosivo e impredecible
longlife	B345/S5	Estructuras que persisten mucho tiempo

```

survive = set()
for part in parts:
    if part.startswith("B"): born = {int(d) for d in part[1:]}
    if part.startswith("S"): survive = {int(d) for d in part[1:]}
return AutomataRule(born, survive)

def get_rule(name_or_string: str) -> AutomataRule:
    """Busca en PRESETS; si no, parsea como regla personalizada."""
    if name_or_string in PRESETS:
        return PRESETS[name_or_string]
    return parse_rule(name_or_string)

```

4.4. Puntos para exponer

1. **Paralelismo de datos:** el tablero se divide en franjas de filas. Cada CPU calcula su franja sin comunicarse con las demás. Al final se reensambla. Es el mismo algoritmo, pero distribuido.
2. **¿Por qué funciona?:** cada generación depende solo de la generación anterior. No hay dependencia entre franjas durante el cálculo. Cada worker solo lee el tablero completo y escribe en su propio segmento.
3. **Las reglas no son fijas:** el worker recibe `born` y `survive` como parámetros. Cambiar de Conway a HighLife es instantáneo.
4. **11 universos distintos:** mismo patrón, diferentes reglas = comportamientos radicalmente distintos. Demo: glider con Conway → se mueve; con Seeds → muere; con Day&Night → algo completamente diferente.
5. **Cámara y zoom:** la grilla real es de 200×200, pero la pantalla es de 800×800. La cámara permite explorar el tablero virtual.

5. Apéndice: referencia rápida de comandos

5.1. Ejecutar cada versión

```
# Versión base (GUI)
python juego_vida.py

# Versión optimizada (demo automática)
python juego_vida_optimo.py

# Versión paralela (GUI con cámara)
python juego_vida_proyecto.py
```

5.2. Cargar patrones desde código

```
from juego_vida_optimo import GameOfLifeOptimo
g = GameOfLifeOptimo()

# Patrones disponibles
g.list_pattern_names()

# Cargar y ejecutar
g.load_pattern("glider")
g.run_n_steps(10)
print(g.get_cells())

# Cambiar regla
g.set_rule("highlife")
g.load_pattern("r_pentomino")
g.run_n_steps(500)
```

5.3. Controles unificados (versiones GUI)

Tecla	Acción
Espacio	Pausar / reanudar
C	Limpiar grilla
1–9	Cargar patrón precargado
P	Imprimir catálogo en consola
R	Cambiar regla de autómata
S	Guardar patrón actual a YAML
Click	Pintar / borrar célula
Rueda	Zoom (solo v3)
WASD	Mover cámara (solo v3)